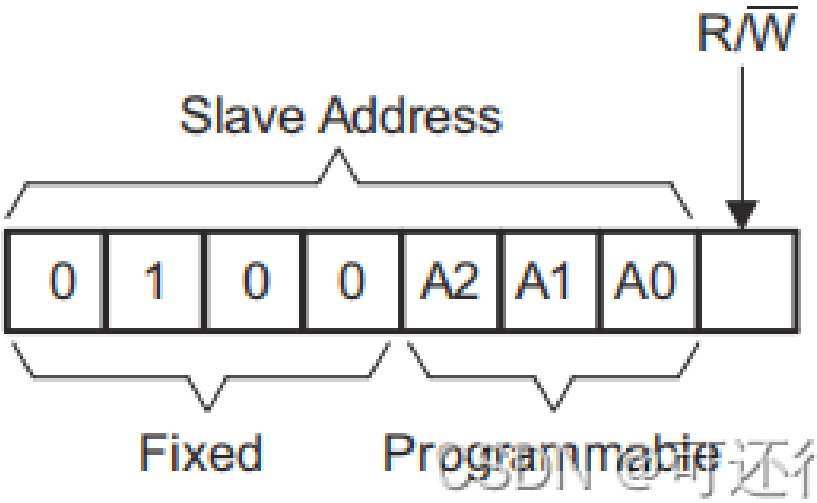


【PCA9555详细学习及代码】

本文详细讲诉了PCA9555芯片的功能以及使用情况，并给出了实例代码提供学习参考

可还行吧！
10031人浏览 · 2023-09-17 14:21:46
可还行吧！ · 2023-09-17 14:21:46 发布

PCA9555详细学习及代码

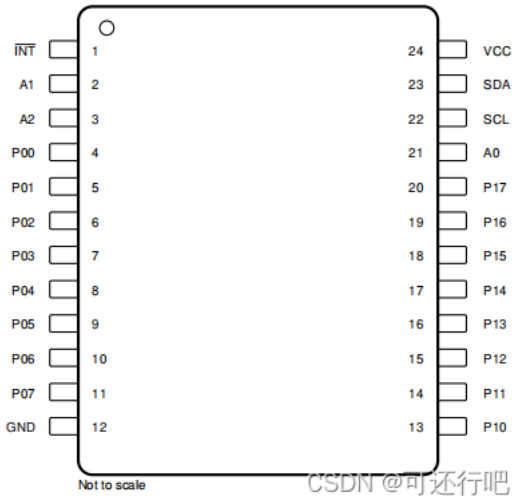
本文参考
https://blog.csdn.net/TheOneZn/article/details/122723644
https://blog.csdn.net/LH_SMD/article/details/121354625

PCA9555芯片介绍

PCA9555是一款16位通用输入/输出（GPIO）扩展器，具有中断和弱上拉电阻，适用于I2c总线/SMBus应用。它具有以下特点：
1.PCA9555由两个8位配置（输入或输出选择），输入端口，输出端口和极性反转（高电平有效或低电平有效运行）寄存器组成。
2.在加电时I /O被配置为输入，系统主控制器可以通过写入I /O配置位将I /O启动为输入或输出。
3.每一个输入或者输出的数据被保存在相应的输入或者输出寄存器内。
4.输入端口寄存器的极性可借助极性反转寄存器进行转换。
5.所有寄存器都可由系统主控器读取。
芯片引脚及功能描述：

可还行吧！
@weixin_49275308
关注
已为社区贡献1条内容

开源项目
OpenTenBase
GreatSQL
MiniBlink
Carsmos
OpenKona
ZTDBP
UBML
RT-TKern
HyperBench
UBSICE
Vearch
源师兄
铜锁/Tongsuo
狮偶
openEuler
TobudOS
hapjs
AliOS Things
PikiwiDB
OpenHarmony

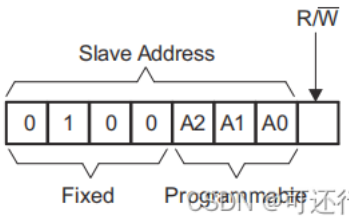


PIN			DESCRIPTION
NAME	SSOP (DB), QSOP (DBQ), TSSOP (PW), AND TVSOP (DGV)	QFN (RGE)	
$\overline{INT}$	1	22	Interrupt output. Connect to V_{CC} through a pullup resistor.
A1	2	23	Address input 1. Connect directly to V_{CC} or ground.
A2	3	24	Address input 2. Connect directly to V_{CC} or ground.
P00	4	1	P-port input/output. Push-pull design structure.
P01	5	2	P-port input/output. Push-pull design structure.
P02	6	3	P-port input/output. Push-pull design structure.
P03	7	4	P-port input/output. Push-pull design structure.
P04	8	5	P-port input/output. Push-pull design structure.
P05	9	6	P-port input/output. Push-pull design structure.
P06	10	7	P-port input/output. Push-pull design structure.
P07	11	8	P-port input/output. Push-pull design structure.
GND	12	9	Ground
P10	13	10	P-port input/output. Push-pull design structure.
P11	14	11	P-port input/output. Push-pull design structure.
P12	15	12	P-port input/output. Push-pull design structure.
P13	16	13	P-port input/output. Push-pull design structure.
P14	17	14	P-port input/output. Push-pull design structure.
P15	18	15	P-port input/output. Push-pull design structure.
P16	19	16	P-port input/output. Push-pull design structure.
P17	20	17	P-port input/output. Push-pull design structure.
A0	21	18	Address input 0. Connect directly to V_{CC} or ground.
SCL	22	19	Serial clock bus. Connect to V_{CC} through a pullup resistor.
SDA	23	20	Serial data bus. Connect to V_{CC} through a pullup resistor.
V_{CC}	24	21	Supply voltage

PCA9555详细学习

1.INT引脚功能：当任何输入状态与其对应的状态不同时，PCA9555开漏中断(INT)引脚输出（只对被配置为输入模式的引脚产生反应），用于向单片机指示输入状态已更改。INT可以连接到微控制器的中断输入端，通过在这条线发送中断信号给单片机，如果端口上有传入数据，远程I/O可以通知微控制器，而不必通过通信I2C总线。因此PCA9555可以保持一个简单的从设备。

2.PCA9555芯片采用iic的方式进行通信，其地址如下：
其前四位为固定位，后三位A0，A1，A2为可编程位，用于设置地址。从地址的最后一位定义了要执行的操作(读或写)。高(1)表示读操作，低(0)表示写操作。



3.控制寄存器和命令字节
当地址位正确应答之后，主机应该发送一个控制字节，控制字节将会存PCA9555的控制寄存器中，其中B0，B1，B2三位，将会定义操作以及内部的寄存器(输入、输出、极性反转或配置)，控制寄存器能被读以及写，控制字节只有写的时候才发送。一旦发送了一个命令字节，被寻址的寄存器将继续被读访问，直到已发送新的命令字节。command

其实就相当于相应的控制寄存器地址，比如000就是访问input port0的寄存器

图 8-7. Control Register Bits

7	6	5	4	3	2	1	0
0	0	0	0	0	B2	B1	B0

表 8-3. Command Byte

CONTROL REGISTER BITS			COMMAND BYTE (HEX)	REGISTER	PROTOCOL	POWER-UP DEFAULT
B2	B1	B0				
0	0	0	0x00	Input Port 0	Read byte	xxxx xxxx
0	0	1	0x01	Input Port 1	Read byte	xxxx xxxx
0	1	0	0x02	Output Port 0	Read/write byte	1111 1111
0	1	1	0x03	Output Port 1	Read/write byte	1111 1111
1	0	0	0x04	Polarity Inversion Port 0	Read/write byte	0000 0000
1	0	1	0x05	Polarity Inversion Port 1	Read/write byte	0000 0000
1	1	0	0x06	Configuration Port 0	Read/write byte	1111 1111
1	1	1	0x07	Configuration Port 1	Read/write byte	1111 1111

4.PCA9555芯片具有四种内部寄存器：

输入寄存器：输入端口寄存器(寄存器0和1)反映引脚的传入逻辑电平，而不管是否引脚由配置寄存器定义为输入或输出。它只作用于读操作。写入这些寄存器没有作用。默认值X由外部应用的逻辑级别决定。在进行读操作之前，发送一个带有命令字节的写传输，以指示I2C设备接下来将访问输入端口寄存器。

表 8-4. Registers 0 And 1 (Input Port Registers)

Bit	I0.7	I0.6	I0.5	I0.4	I0.3	I0.2	I0.1	I0.0
Default	X	X	X	X	X	X	X	X
Bit	I1.7	I1.6	I1.5	I1.4	I1.3	I1.2	I1.1	I1.0
Default	X	X	X	X	X	X	X	X

输出寄存器：输出端口寄存器(寄存器2和3)显示定义为输出的引脚的传出逻辑电配置寄存器。这个寄存器中的位值对定义为输入的引脚没有影响。反过来，从这里读寄存器反映控制输出选择的触发器中的值，而不是实际引脚值。

表 8-5. Registers 2 And 3 (Output Port Registers)

Bit	O0.7	O0.6	O0.5	O0.4	O0.3	O0.2	O0.1	O0.0
Default	1	1	1	1	1	1	1	1
Bit	O1.7	O1.6	O1.5	O1.4	O1.3	O1.2	O1.1	O1.0
Default	1	1	1	1	1	1	1	1

极性反转寄存器：极性反转寄存器(寄存器4和5)允许定义为输入的引脚极性反转配置寄存器。向该寄存器里某一位写入1，表明该引脚在输入状态的时候输入极性会反转，写入0则不起作用。

表 8-6. Registers 4 And 5 (Polarity Inversion Registers)

Bit	N0.7	N0.6	N0.5	N0.4	N0.3	N0.2	N0.1	N0.0
Default	0	0	0	0	0	0	0	0
Bit	N1.7	N1.6	N1.5	N1.4	N1.3	N1.2	N1.1	N1.0
Default	0	0	0	0	0	0	0	0

配置寄存器：配置寄存器(寄存器6和7)配置I/O引脚的方向。如果这个寄存器中的位是设置为1，对应的端口引脚作为高阻抗输出驱动器的输入启用。如果在这一点寄存器清除为0，相应的端口引脚作为输出启用。

表 8-7. Registers 6 And 7 (Configuration Registers)

Bit	C0.7	C0.6	C0.5	C0.4	C0.3	C0.2	C0.1	C0.0
Default	1	1	1	1	1	1	1	1
Bit	C1.7	C1.6	C1.5	C1.4	C1.3	C1.2	C1.1	C1.0
Default	1	1	1	1	1	1	1	1

5.通过上面的介绍，PCA9555的功能应该有所了解了，下面来看下然后使用这个芯片：

例1：主机发送从设备地址，从设备地址匹配，从设备发送ack，主机接收应答后发生命令字节，比如发送0x02，等待从机应答后，即可向输出寄存器写入数据改变拓展IO引脚输出。如果命令直接写入的配置寄存器的地址，则可配置IO口的输入输出状态。（在写入输出状态前因先将IO口通过配置寄存器设置为输出状态）

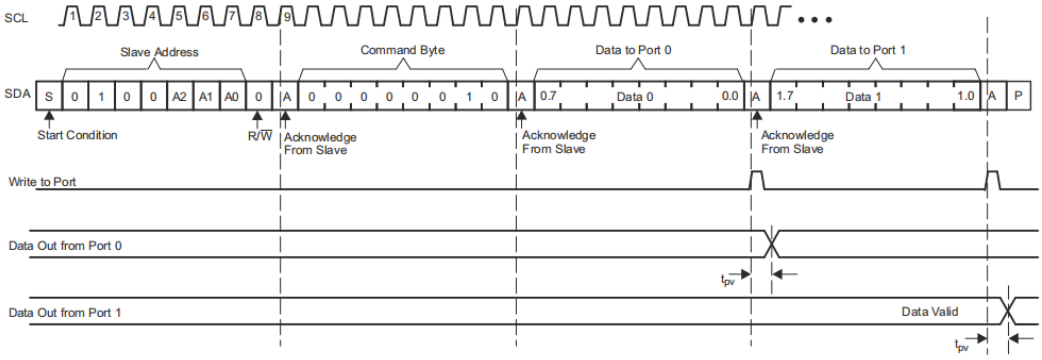


图 8-8. Write To Output Port Registers

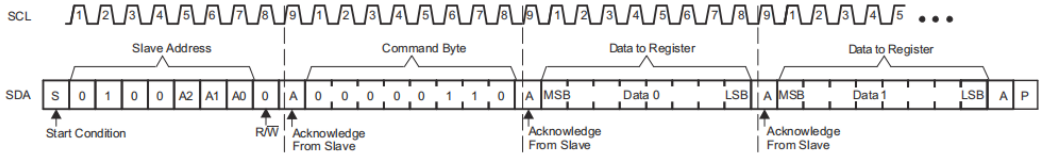
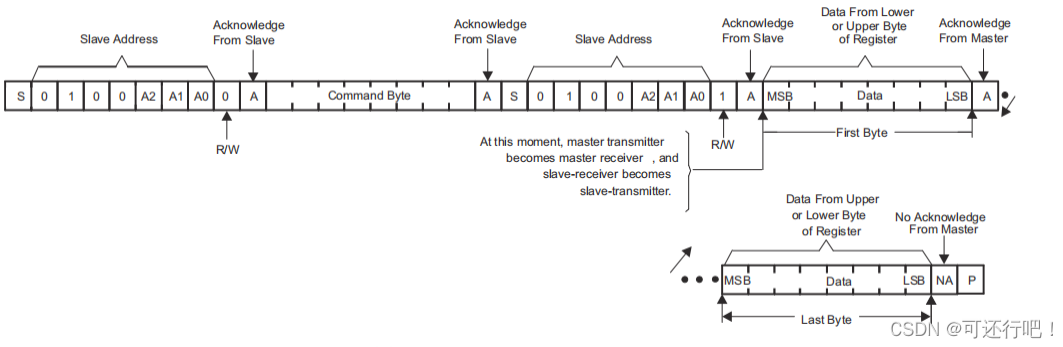


图 8-9. Write To Configuration Registers

例2：主机发送从设备地址（主要此时起始信号为读写位为写状态，因为等下得得写入命令字节），从设备地址匹配，从设备发送ack，主机发送命令字节，接收应答后，重新发送设备地址（此时起始信号为读写位为读取状态），等待应答主机即可读取从机发送过来得数据。



PCA9555代码实例

头文件：

```
#ifndef USER_APP_PCA9555_H_
#define USER_APP_PCA9555_H_

#include "stm32f4xx_ll_gpio.h"
/************************************************IIC 驱动部门*****************************************
#define IIC_SCL_GPIO_PORT  GPIOB
#define IIC_SCL_GPIO_PIN   LL_GPIO_PIN_7

#define IIC_SDA_GPIO_PORT  GPIOB
#define IIC_SDA_GPIO_PIN   LL_GPIO_PIN_8

#define IIC_SCL_SET_GPIO_OUTPUT_STATUS(status)      if(status == 1)   LL_GPIO_SetOutputPin(IIC_SCL_GPIO_
                                                    else if(status == 0)  LL_GPIO_ResetOutputPin(IIC_SCL

#define IIC_SDA_SET_GPIO_OUTPUT_STATUS(status)     if(status == 1)   LL_GPIO_SetOutputPin(IIC_SDA_GPIO_P
                                                    else if(status == 0)  LL_GPIO_ResetOutputPin(IIC_SDA

#define IIC_SDA_GET_GPIO_INPUT_STATUS               LL_GPIO_IsInputPinSet(IIC_SDA_GPIO_PORT, IIC_SDA_GPI


void IIC_gpio_init(void);
void IIC_start(void);
void IIC_stop(void);
uint8_t IIC_wait_ack(void);
void IIC_ack(void);
```

```
void IIC_nack(void);
void IIC_send_byte(uint8_t txd);
void IIC_send_byte(uint8_t txd);
uint8_t IIC_read_byte(unsigned char ack);
/*************************************************IIC 驱动 END*****************************

/*************************************************PCA9555 驱动*****************************
#define SUCCESS 0
#define ERROR   1

#define SLAVE_ADDR0   0x40

#define HOST_WRITE_COMMAND 0x00
#define HOST_READ_COMMAND  0x01

#define INPUT_PORT_REGISTER0                    0x00   /* 输入端口寄存器0，负责IO00-IO07 */
#define INPUT_PORT_REGISTER1                    0x01   /* 输入端口寄存器1，负责IO10-IO17 */
#define OUTPUT_PORT_REGISTER0                   0x02   /* 输入端口寄存器0，负责IO00-IO07 */
#define OUTPUT_PORT_REGISTER1                   0x03   /* 输入端口寄存器1，负责IO10-IO17 */
#define POLARITY_INVERSION_PORT_REGISTER0       0x04   /* 极性反转端口寄存器0，负责IO00-IO07 */
#define POLARITY_INVERSION_PORT_REGISTER1       0x05   /* 极性反转端口寄存器1，负责IO10-IO17 */
#define CONFIG_PORT_REGISTER0                   0x06   /* 输入端口寄存器0，负责IO00-IO07 */
#define CONFIG_PORT_REGISTER1                   0x07   /* 输入端口寄存器1，负责IO10-IO17 */

#define GPIO_PORT0 0
#define GPIO_PORT1 1

#define GPIO_0 0x01
#define GPIO_1 0x02
#define GPIO_2 0x04
#define GPIO_3 0x08
#define GPIO_4 0x10
#define GPIO_5 0x20
#define GPIO_6 0x40
#define GPIO_7 0x80


#define PCA9555_WAIT_IS_RETURN_SUCCESS(flag, tips)    if(flag != SUCCESS)\
                                                        { \
                                                            printf("%s", tips); \
                                                            return ERROR;\
                                                        }

void pca9555_init(void);
void pca9555_read_wtite_test(void);
void pca9555_set_output_mode_all(uint8_t slave_num, uint8_t gpio_port);
void pca9555_set_output_mode(uint8_t slave_num, uint8_t gpio_port,  uint8_t gpio_num);
void pca9555_set_input_mode_all(uint8_t slave_num, uint8_t gpio_port);
void pca9555_set_gpio_output_status(uint8_t slave_num, uint8_t gpio_port,  uint8_t gpio_num, uint8_t sta
void pca9555_set_input_mode(uint8_t slave_num, uint8_t gpio_port,  uint8_t gpio_num);
void pca9555_set_input_mode_all(uint8_t slave_num, uint8_t gpio_port);
void pca9555_set_output_mode_all(uint8_t slave_num, uint8_t gpio_port);
uint8_t pca9555_get_gpio_status(uint8_t slave_num, uint8_t gpio_port,  uint8_t gpio_num);
uint8_t pca9555_read_byte(uint8_t slave_num, uint8_t addr, uint8_t read_register_data, uint8_t *read_dat
uint8_t pca9555_write_byte(uint8_t addr, uint8_t command, uint8_t write_register_data);

#endif /* USER_APP_PCA9555_H_ */
```

源文件：

```
#include "hardware/pca9555.h"
#include "main.h"
#include "hardware/timer.h"
#include <stdio.h>

void IIC_gpio_init(void)
```

```
{
    LL_AHB1_GRP1_EnableClock(LL_AHB1_GRP1_PERIPH_GPIOC);    /* 使能GPIO时钟 */

    LL_GPIO_SetPinMode(IIC_SCL_GPIO_PORT, IIC_SCL_GPIO_PIN, LL_GPIO_MODE_OUTPUT);//设置为推挽输出，初始化高
    LL_GPIO_SetPinOutputType(IIC_SCL_GPIO_PORT, IIC_SCL_GPIO_PIN,LL_GPIO_OUTPUT_PUSHPULL);//设置为推挽输出
    LL_GPIO_SetOutputPin(IIC_SCL_GPIO_PORT, IIC_SCL_GPIO_PIN);

    LL_GPIO_SetPinMode(IIC_SDA_GPIO_PORT, IIC_SDA_GPIO_PIN, LL_GPIO_MODE_OUTPUT);//设置为推挽输出，初始化高
    LL_GPIO_SetPinOutputType(IIC_SDA_GPIO_PORT, IIC_SDA_GPIO_PIN,LL_GPIO_OUTPUT_PUSHPULL);
    LL_GPIO_SetOutputPin(IIC_SDA_GPIO_PORT, IIC_SDA_GPIO_PIN);
}


static void IIC_SDA_set_output(void)
{
    LL_GPIO_SetPinMode(IIC_SDA_GPIO_PORT, IIC_SDA_GPIO_PIN, LL_GPIO_MODE_OUTPUT);
    LL_GPIO_SetPinOutputType(IIC_SDA_GPIO_PORT, IIC_SDA_GPIO_PIN, LL_GPIO_OUTPUT_PUSHPULL);
    LL_GPIO_SetOutputPin(IIC_SDA_GPIO_PORT, IIC_SDA_GPIO_PIN);
}

static void IIC_SDA_set_input(void)
{
    LL_GPIO_SetPinMode(IIC_SDA_GPIO_PORT, IIC_SDA_GPIO_PIN, LL_GPIO_MODE_INPUT);
    LL_GPIO_SetPinPull(IIC_SDA_GPIO_PORT, IIC_SDA_GPIO_PIN, LL_GPIO_PULL_UP);
}

//产生IIC起始信号
void IIC_start(void)
{
    IIC_SDA_set_output(); //sda线输出
    IIC_SDA_SET_GPIO_OUTPUT_STATUS(1);
    IIC_SCL_SET_GPIO_OUTPUT_STATUS(1);
    //sl_udelay_wait(6);
    delay_us(6);
    IIC_SDA_SET_GPIO_OUTPUT_STATUS(0);//START:when CLK is high,DATA change form high to low
    //sl_udelay_wait(6);
    delay_us(6);
    IIC_SCL_SET_GPIO_OUTPUT_STATUS(0);//钳住I2C总线，准备发送或接收数据
}

//产生IIC停止信号
void IIC_stop(void)
{
    IIC_SDA_set_output();//sda线输出
    IIC_SCL_SET_GPIO_OUTPUT_STATUS(0);
    IIC_SDA_SET_GPIO_OUTPUT_STATUS(0);//STOP:when CLK is high DATA change form low to high
    //sl_udelay_wait(6);
    delay_us(6);
    IIC_SCL_SET_GPIO_OUTPUT_STATUS(1);
    IIC_SDA_SET_GPIO_OUTPUT_STATUS(1);//发送I2C总线结束信号
    //sl_udelay_wait(6);
    delay_us(6);
}

//等待应答信号到来
//返回值：1，接收应答失败
//        0，接收应答成功
uint8_t IIC_wait_ack(void)
{
    uint8_t ucErrTime=0;
    IIC_SDA_set_input();  //SDA设置为输入
    IIC_SDA_SET_GPIO_OUTPUT_STATUS(1);
    //sl_udelay_wait(1);
    delay_us(1);
    IIC_SCL_SET_GPIO_OUTPUT_STATUS(1);
    //sl_udelay_wait(1);
    delay_us(1);

    while(IIC_SDA_GET_GPIO_INPUT_STATUS)
    {
        ucErrTime++;
```

```
        //sl_udelay_wait(1);
        delay_us(1);
        if(ucErrTime>250)
        {
            IIC_stop();
            printf("return 1\r\n");
            return 1;
        }
    }
    IIC_SCL_SET_GPIO_OUTPUT_STATUS(0);//时钟输出0
    return 0;
}

//产生ACK应答
 void IIC_ack(void)
{
    IIC_SCL_SET_GPIO_OUTPUT_STATUS(0);
    IIC_SDA_set_output();
    IIC_SDA_SET_GPIO_OUTPUT_STATUS(1);
    //sl_udelay_wait(5);
    delay_us(5);
    IIC_SCL_SET_GPIO_OUTPUT_STATUS(1);
    //sl_udelay_wait(5);
    delay_us(5);
    IIC_SCL_SET_GPIO_OUTPUT_STATUS(0);
}

//不产生ACK应答
void IIC_nack(void)
{
    IIC_SCL_SET_GPIO_OUTPUT_STATUS(0);
    IIC_SDA_set_output();
    IIC_SDA_SET_GPIO_OUTPUT_STATUS(1);
    //sl_udelay_wait(5);
    delay_us(5);
    IIC_SCL_SET_GPIO_OUTPUT_STATUS(1);
    //sl_udelay_wait(5);
    delay_us(5);
    IIC_SCL_SET_GPIO_OUTPUT_STATUS(0);
}

//IIC发送一个字节(高位先发)
//返回从机有无应答
//1，有应答
//0，无应答
void IIC_send_byte(uint8_t txd)
{
    uint8_t t;
    IIC_SDA_set_output();
    IIC_SCL_SET_GPIO_OUTPUT_STATUS(0);//拉低时钟开始数据传输
    for(t=0;t<8;t++)
    {
        //printf("\r\n1 GPIO_PinInGet(IIC_SDA_GPIO_PORT, IIC_SDA_GPIO_PIN) = %d\r\n",  GPIO_PinInGet(IIC
        IIC_SDA_SET_GPIO_OUTPUT_STATUS((txd&0x80)>>7);
        txd<<=1;
        delay_us(4);
        IIC_SCL_SET_GPIO_OUTPUT_STATUS(1);
        delay_us(4);
        IIC_SCL_SET_GPIO_OUTPUT_STATUS(0);
        delay_us(4);
    }
}

//读1个字节，ack=1时，发送ACK，ack=0，发送nACK(高位先接收)
uint8_t IIC_read_byte(unsigned char ack)
{
    unsigned char i,receive=0;
    IIC_SDA_set_input();//SDA设置为输入
    for(i=0;i<8;i++)
    {
        IIC_SCL_SET_GPIO_OUTPUT_STATUS(0);
```

```
        delay_us(4);
        IIC_SCL_SET_GPIO_OUTPUT_STATUS(1);
        receive<<=1;
        if(IIC_SDA_GET_GPIO_INPUT_STATUS)receive++;
        //printf("\r\n2 GPIO_PinInGet(IIC_SDA_GPIO_PORT, IIC_SDA_GPIO_PIN) = %d\r\n",  GPIO_PinInGet(IIC
        delay_us(4);
    }
    if (!ack)
        IIC_nack();//发送nACK
    else
        IIC_ack(); //发送ACK
    return receive;
}



void pca9555_init(void)
{
    IIC_gpio_init();
}


//pca9555写寄存器值
uint8_t pca9555_write_byte(uint8_t addr, uint8_t command, uint8_t write_register_data)
{
    uint8_t ret = 1;
    IIC_start();
    IIC_send_byte(addr);    //写从机地址
    ret =  IIC_wait_ack();
    PCA9555_WAIT_IS_RETURN_SUCCESS(ret, "[ERROR] wait slave ack error!\r\n");

    IIC_send_byte(command); //写要写的寄存器地址
    ret =  IIC_wait_ack();
    PCA9555_WAIT_IS_RETURN_SUCCESS(ret, "[ERROR] wait slave ack error!\r\n");

    IIC_send_byte(write_register_data);//写入数据
    ret =  IIC_wait_ack();
    PCA9555_WAIT_IS_RETURN_SUCCESS(ret, "[ERROR] wait slave ack error!\r\n");

    IIC_stop();
    delay_us(10000);
    return SUCCESS;
}

/*
 * pca9555读取寄存器值
 *
 * addr 读取地址
 * read_register_data 要读取的寄存器
 * read_data 读取数据存放地址
 *
 *
 * 返回值：读取成功返回SUCCESS  失败返回ERROR
 *
 * */
uint8_t pca9555_read_byte(uint8_t slave_num, uint8_t addr, uint8_t read_register_data, uint8_t *read_dat
{
    uint8_t ret = 0;
    IIC_start();
    IIC_send_byte(slave_num);           //  写入从机地址
    ret =  IIC_wait_ack();
    PCA9555_WAIT_IS_RETURN_SUCCESS(ret, "[ERROR] wait slave ack error!\r\n");

    IIC_send_byte(read_register_data); //写入要读取的寄存器地址
    ret = IIC_wait_ack();
    PCA9555_WAIT_IS_RETURN_SUCCESS(ret, "[ERROR] wait slave ack error!\r\n");

    IIC_start(); /* 开始接收数据 */
    IIC_send_byte(addr);               //发送从机地址并设置为读取
    ret = IIC_wait_ack();
    PCA9555_WAIT_IS_RETURN_SUCCESS(ret, "[ERROR] wait slave ack error!\r\n");
```

```
    *read_data = IIC_read_byte(0);

    IIC_stop();
    return SUCCESS;
}


/*
 * 设置指定GPIO的模式
 *
 * slave_num  需要操作的从机设备
 * gpio_port  gpio端口  端口0/1
 * gpio_num   哪一个GPIO
 *
 * 返回值：void
 * */
void pca9555_set_output_mode(uint8_t slave_num, uint8_t gpio_port,  uint8_t gpio_num)
{
    uint8_t register_original_data = 0;
    if(gpio_port > 1 || gpio_num > 0x80)  return;
    if(gpio_port == 0)
    {
        pca9555_read_byte(slave_num, slave_num  | HOST_READ_COMMAND, CONFIG_PORT_REGISTER0, &register_or
        pca9555_write_byte(slave_num | HOST_WRITE_COMMAND, CONFIG_PORT_REGISTER0, register_original_data
    }
    else if(gpio_port == 1)
    {
        pca9555_read_byte(slave_num, slave_num  | HOST_READ_COMMAND, CONFIG_PORT_REGISTER1, &register_or
        pca9555_write_byte( slave_num | HOST_WRITE_COMMAND, CONFIG_PORT_REGISTER1, register_original_dat
    }
}

void pca9555_set_output_mode_all(uint8_t slave_num, uint8_t gpio_port)
{
    uint8_t register_original_data = 0x00;
    if(gpio_port > 1)  return;
    if(gpio_port == 0)
    {
        pca9555_write_byte(slave_num | HOST_WRITE_COMMAND, CONFIG_PORT_REGISTER0, register_original_data
    }
    else if(gpio_port == 1)
    {
        pca9555_write_byte( slave_num | HOST_WRITE_COMMAND, CONFIG_PORT_REGISTER1, register_original_dat
    }
}

/*
 * 设置GPIO为输入模式
 *
 * slave_num  需要操作的从机设备
 * gpio_port  gpio端口  端口0/1
 * gpio_num   哪一个GPIO
 *
 * 返回值：void
 **/
void pca9555_set_input_mode(uint8_t slave_num, uint8_t gpio_port,  uint8_t gpio_num)
{
    uint8_t register_original_data = 0;
    if(gpio_port > 1 || gpio_num > 0x80)  return;
    if(gpio_port == 0)
    {
        pca9555_read_byte(slave_num, slave_num  | HOST_READ_COMMAND, CONFIG_PORT_REGISTER0, &register_or
        pca9555_write_byte(slave_num | HOST_WRITE_COMMAND, CONFIG_PORT_REGISTER0, register_original_data
    }
    else if(gpio_port == 1)
    {

        pca9555_read_byte(slave_num, slave_num  | HOST_READ_COMMAND, CONFIG_PORT_REGISTER1, &register_or

        pca9555_write_byte(slave_num | HOST_WRITE_COMMAND, CONFIG_PORT_REGISTER1, register_original_data
```

```
    }
}

void pca9555_set_input_mode_all(uint8_t slave_num, uint8_t gpio_port)
{
    uint8_t register_original_data = 0xff;
    if(gpio_port > 1 )  return;
    if(gpio_port == 0)
    {
        pca9555_write_byte(slave_num | HOST_WRITE_COMMAND, CONFIG_PORT_REGISTER0, register_original_data
    }
    else if(gpio_port == 1)
    {
        pca9555_write_byte(slave_num | HOST_WRITE_COMMAND, CONFIG_PORT_REGISTER1, register_original_data
    }
}

/*
 * 设置GPIO输出状态
 *
 * slave_num  需要操作的从机设备
 * gpio_port  gpio端口  端口0/1
 * gpio_num   哪一个GPIO
 * status     输出状态
 *
 * 返回值：void
 **/
void pca9555_set_gpio_output_status(uint8_t slave_num, uint8_t gpio_port,  uint8_t gpio_num, uint8_t sta
{
    uint8_t register_original_data = 0;
    if(gpio_port > 1 || gpio_num > 0x80)  return;
    if(gpio_port == 0)
    {
        pca9555_read_byte(slave_num, slave_num  | HOST_READ_COMMAND, OUTPUT_PORT_REGISTER0, &register_or
        if(status == 1)
        {
            pca9555_write_byte(slave_num | HOST_WRITE_COMMAND, OUTPUT_PORT_REGISTER0, register_original_
        }
        else
        {
            pca9555_write_byte(slave_num | HOST_WRITE_COMMAND, OUTPUT_PORT_REGISTER0, register_original_
        }
    }
    else if(gpio_port == 1)
    {
        pca9555_read_byte(slave_num, slave_num  | HOST_READ_COMMAND, OUTPUT_PORT_REGISTER1, &register_or
        if(status == 1)
        {
            pca9555_write_byte(slave_num | HOST_WRITE_COMMAND, OUTPUT_PORT_REGISTER1, register_original_
        }
        else
        {
            pca9555_write_byte(slave_num | HOST_WRITE_COMMAND, OUTPUT_PORT_REGISTER1, register_original_
        }
    }
}

/*
 * 获取GPIO状态
 *
 * slave_num  需要操作的从机设备
 * gpio_port  gpio端口  端口0/1
 * gpio_num   哪一个GPIO
 *
 * 返回值：GPIO状态
 **/
uint8_t pca9555_get_gpio_status(uint8_t slave_num, uint8_t gpio_port,  uint8_t gpio_num)
{
    uint8_t register_original_data = 0;
    uint8_t gpio_status = 0;
    if(gpio_port > 1 || gpio_num > 0x80)
```

```
    {
        printf("[ERROR] gpio_port > 1 || gpio_num > 0x80\r\n");
        return 2;
    }
    if(gpio_port == 0)
    {

        pca9555_read_byte(slave_num, slave_num  | HOST_READ_COMMAND, INPUT_PORT_REGISTER0, &register_ori

    }
    else if(gpio_port == 1)
    {
        pca9555_read_byte(slave_num, slave_num  | HOST_READ_COMMAND, INPUT_PORT_REGISTER1, &register_ori
    }
    switch(gpio_num)
    {
        case 0x01:  gpio_status = register_original_data & gpio_num;break;
        case 0x02:  gpio_status = (register_original_data & gpio_num) >> 1;break;
        case 0x04:  gpio_status = (register_original_data & gpio_num) >> 2;break;
        case 0x08:  gpio_status = (register_original_data & gpio_num) >> 3;break;
        case 0x10:  gpio_status = (register_original_data & gpio_num) >> 4;break;
        case 0x20:  gpio_status = (register_original_data & gpio_num) >> 5;break;
        case 0x40:  gpio_status = (register_original_data & gpio_num) >> 6;break;
        case 0x80:  gpio_status = (register_original_data & gpio_num) >> 7;break;
        default:    printf("[ERROR] gpio error!\r\n");
    }
    return gpio_status;
}
```

注：
PCA9555采用IIC通信，协议中涉及的延时函数务必保证精确。
上面代码可直接运行在STM32单片机中，只需要对以下两部点进行修改即可
①GPIO输入输出设置
②精确到us级别的延时函数

c语言　　# 嵌入式硬件

更多推荐

· 人工智能在库存管理中的应用
· dubbo启动报错failed to bind nettyserver on
· 【Windows】redis安装

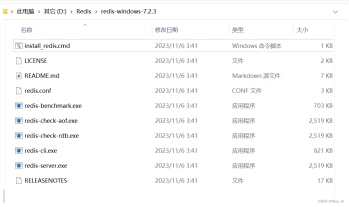
人工智能在库存管理中的应用

开放原子开发者工作坊

dubbo启动报错failed to bind nettyserver on

dubbo报错今天启动项目的时候，关掉了custom服务，<dubbo:consumer check="false"/>并且关掉了spring的elastic-job，<!--<import resource="cloud-elastic-job.xml"/>-->但是还是报错，看了下错误代码，原因是因...

开放原子开发者工作坊



【Windows】redis安装

开放原子开发者工作坊

所有评论(0)

请输入

发表